

OVERVIEW

System architecture

What the system does, how its parts connect, and why it is built this way.

The problem

Restaurants, supermarkets, cafeterias and cloud kitchens order food every morning without knowing how much will be wasted by the evening. This system predicts, **at the start of the day**, whether today's stock for a business and food category will end with **Low, Medium or High waste**. That leaves time to act: order less, run a promotion, or plan a donation.

A model on its own is not enough to be useful in production. It has to be trained reproducibly, checked automatically, served reliably and watched after release. That is what MLOps adds, and each piece below covers one of those jobs.

Architecture diagram

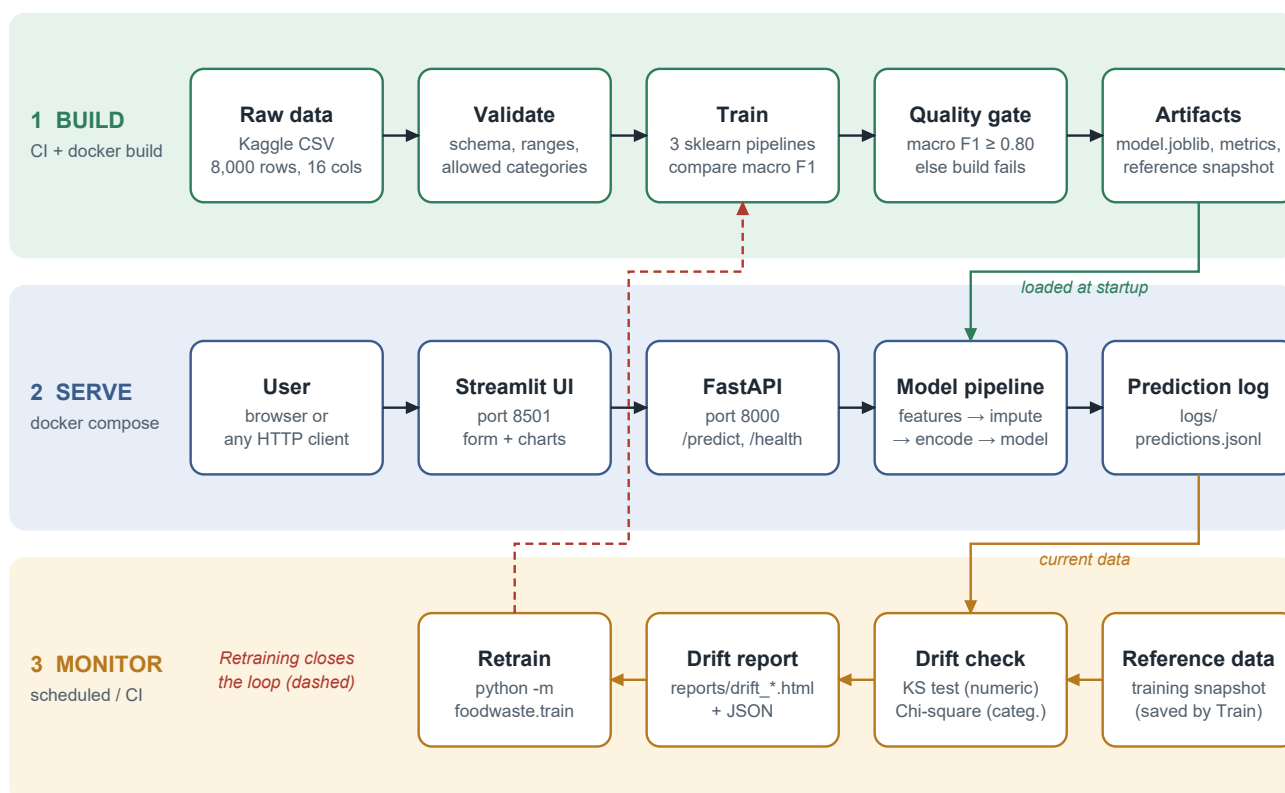


Figure 1 - Three lanes: Build creates a verified model, Serve answers requests, Monitor compares incoming data with the training snapshot and triggers retraining (dashed red) when it drifts.

The three flows

1. Build flow (runs in CI and during docker build)

Raw CSV $\rightarrow$ validation $\rightarrow$ three candidate pipelines trained and compared $\rightarrow$ the best one must reach macro F1 ≥ 0.80 $\rightarrow$ only then are the model, its metrics and a snapshot of the training data saved. If any step fails, nothing is saved and the build goes red.

2. Serve flow (runs in Docker containers)

A user fills in the Streamlit form → the UI sends JSON to FastAPI /predict → FastAPI validates the input, runs the saved pipeline, returns the level with probabilities and the model version → every request and prediction is appended to a log file.

3. Monitor flow (scheduled, or on demand)

The drift check compares new data (the API log, a CSV, or a simulated batch) against the training snapshot, feature by feature. It writes an HTML and JSON report. If drift is found, the model is retrained, which starts the build flow again.

Components and technology

Component	Technology	Responsibility	Code
Config	Python	Paths, feature lists, thresholds in one place	foodwaste/config.py
Validation	pandas	Reject bad data before it reaches the model	foodwaste/data.py
Training	scikit-learn	Pipelines, model comparison, quality gate, artifacts	foodwaste/train.py
Prediction	joblib	Load the model once and predict from raw records	foodwaste/predict.py
API	FastAPI, pydantic	REST endpoints, input schema, request logging	api/main.py
UI	Streamlit	Form, result, model metrics, drift demo	app/streamlit_app.py
Monitoring	SciPy	KS / chi-square drift tests, HTML reports	foodwaste/drift.py
Tests	pytest, httpx	21 automated checks	tests/
Container	Docker, compose	Reproducible runtime; trains at build time	Dockerfile, docker-compose.yml
CI/CD	GitHub Actions	Train, test, drift-check, build, smoke-test on every push	.github/workflows/ci.yml

Folder structure

```

food-waste-mlops/
├── data/                raw dataset + data dictionary (committed)
├── foodwaste/           core package - used by notebook, API, UI, CI and Docker
│   ├── config.py        single source of truth: paths, features, thresholds
│   ├── data.py          load, validate, feature engineering
│   ├── train.py         train + compare + quality gate + save artifacts
│   ├── predict.py       load model, predict from raw records
│   └── drift.py         drift detection + HTML/JSON reports
├── api/main.py          FastAPI service
├── app/streamlit_app.py Streamlit UI
├── tests/               pytest suite (data, model, api, drift)
├── notebooks/walkthrough.ipynb
├── models/ logs/ reports/ generated - not committed to git
├── Dockerfile docker-compose.yml .dockerignore
├── .github/workflows/ci.yml
├── requirements.txt requirements-dev.txt pytest.ini README.md

```

Design principles

- **One codebase, many entry points.** The notebook, API, UI, tests, CI and Docker all import the same foodwaste package, so there is never a 'notebook version' and a 'production version' that behave differently.
- **All preprocessing lives inside the model pipeline.** Feature engineering, imputation and encoding are saved together with the model. The API sends raw fields, so training and serving cannot drift apart.
- **No data leakage.** Only information available in the morning is used as input.
- **Fail loudly and early.** Bad data, a weak model, a broken API or a failing container all stop the pipeline instead of reaching users.
- **Artifacts are rebuilt, not stored.** Models, logs and reports are excluded from git; any machine can recreate them from code + data.
- **Every prediction is traceable.** Responses carry a model version; metrics.json records the data hash, library version and scores of the run that produced it.